Kapnet Over Bitcoin, Volume IV

Worked Protocol Scenarios, Actor Flows, and Real Operational Use

A scenario-heavy companion focused on nodes, relays, rooms, guilds, DAOs, members, submissions, and time

This volume shifts from ontology to motion. It treats Kapnet not as a list of primitives but as a sequence of lived flows: a member joins, a submission arrives, relays compete, a room gates discussion, a guild asks for expert attestation, a DAO mutates, a knot race occurs, nodes disagree temporarily, archives crystallize, and replay stabilizes the result.

March 24, 2026

Contents

Abstract **2**

1 Purpose of This Volume **3**

2 Actor Model Used Throughout the Scenarios **3**

2.1 End users and members . . . 3

2.2 Nodes . . . 4

2.3 Relays . . . 4

2.4 Rooms . . . 4

2.5 Guilds . . . 4

2.6 DAOs . . . 4

2.7 Providers . . . 5

3 Time Semantics Used Throughout **5**

3.1 Wall-clock time . . . 5

3.2 Relay arrival time . . . 5

3.3 Base-block affinity time . . . 5

3.4 Braid time . . . 5

3.5 Knot time . . . 6

4 A Generic Scenario Template **6**

5 Scenario I: A Member Posts Into a Weakwork-Gated Room **6**

5.1 Simple room post under real relay policy . . . 6

5.1.1 Actors . . . 6

5.1.2 Initial state . . . 7

5.1.3 Trigger . . . 7

5.1.4 Illustrative packet body . . . 7

5.1.5 Time layers . . . 8

5.1.6 Relay behavior . . . 8

5.1.7 Replay result . . . 8
5.1.8 Consequences . . . 8
5.1.9 Failure variant . . . 8

6 Scenario II: A Guild Requests Expert Attestation Before Publication 9

6.1 Guild editorial workflow with subject-matter expertise . . . 9
6.1.1 Actors . . . 9
6.1.2 Initial state . . . 9
6.1.3 Flow . . . 9
6.1.4 Timeline table . . . 10
6.1.5 Replay result . . . 10
6.1.6 Consequences . . . 10

7 Scenario III: A New Room Is Created Under a Guild Constitution 10

7.1 Room creation with bounded authority . . . 10
7.1.1 Actors . . . 10
7.1.2 Flow . . . 11
7.1.3 Real significance . . . 11
7.1.4 Consequences . . . 11

8 Scenario IV: Membership Admission With Bond, Sponsorship, and Cooldown 11

8.1 Guild membership is not just a boolean . . . 12
8.1.1 Actors . . . 12
8.1.2 Flow . . . 12
8.1.3 Time importance . . . 12
8.1.4 Consequences . . . 12

9 Scenario V: A Treasury-Bonded Proposal in a DAO-Controlled Guild 12

9.1 Economic seriousness for consequential proposals . . . 13
9.1.1 Actors . . . 13
9.1.2 Flow . . . 13
9.1.3 Why this matters . . . 13

10 Scenario VI: A Dispute Over Moderation Escalates From Room to Guild to DAO 13

10.1 Moderation becomes constitutional when replayable . . . 13

10.1.1 Actors . . . 14

10.1.2 Flow . . . 14

10.1.3 Consequences . . . 14

11 Scenario VII: A Node Comes Back Online After Missing a Knot Cycle 14

11.1 Replay after outage . . . 14

11.1.1 Actors . . . 14

11.1.2 Flow . . . 15

11.1.3 Why this matters . . . 15

11.1.4 Second-order consequence . . . 15

12 Scenario VIII: Knot Race During Relay Fragmentation 15

12.1 Checkpoint politics under imperfect network conditions . . . 15

12.1.1 Actors . . . 15

12.1.2 Flow . . . 16

12.1.3 Consequences . . . 16

13 Scenario IX: A Publication Pipeline Across Rooms, Guild, Archive, and Indexer 16

13.1 From room discussion to durable public object . . . 16

13.1.1 Flow overview . . . 16

13.1.2 Why this is a real use case . . . 17

14 Scenario X: Cross-Room Working Group Coordination 17

14.1 One project, many rooms, one guild memory . . . 17

14.1.1 Actors . . . 17

14.1.2 Flow . . . 17

14.1.3 Consequence . . . 18

15 Scenario XI: Multi-Tip Coordination During Divergent Policy Worlds 18

15.1 One institution watches several timechains . . . 18

15.1.1 Actors . . . 18
15.1.2 Flow . . . 18
15.1.3 Why this is meaningful . . . 18
15.1.4 Third-order effect . . . 18

16 Scenario XII: Provider Contract for High-Retention Archival Service 19
16.1 Service contracts become protocol-visible . . . 19
16.1.1 Actors . . . 19
16.1.2 Flow . . . 19
16.1.3 Consequence . . . 19

17 Scenario XIII: Emergency Room Fork Under Constitutional Conflict 19
17.1 Forkability as institutional safety valve . . . 20
17.1.1 Flow . . . 20
17.1.2 Consequences . . . 20

18 Scenario XIV: A Local Family or Small Firm Uses Kapnet as Internal Coordination Memory 20
18.1 Not every use case is a public guild . . . 20
18.1.1 Use pattern . . . 20
18.1.2 Why this matters . . . 21

19 Scenario XV: A Disaster-Response Guild Coordinates Resources Under Time Pressure 21
19.1 High urgency, bounded trust, many actors . . . 21
19.1.1 Why Kapnet fits . . . 21
19.1.2 Consequence . . . 21

20 Scenario XVI: A Research Lab Uses Kapnet for Peer Review and Priority Claims 21
20.1 Scientific coordination without platform opacity . . . 22
20.1.1 Flow . . . 22
20.1.2 Importance . . . 22

21 Scenario XVII: Node Operator Sells Weakwork-Accelerated Relay to a Guild 22

21.1 Small operators can sell real service . . . 22

21.1.1 Consequence . . . 22

22 Scenario XVIII: A Full Institutional Publication and Governance Cycle 22

22.1 Everything at once . . . 23

22.1.1 Narrative outline . . . 23

22.1.2 Why this scenario matters . . . 23

23 Application Atlas 23

23.1 Publishing and editorial institutions . . . 23

23.2 Research labs and peer review systems . . . 24

23.3 Professional guilds and trade bodies . . . 24

23.4 Firms and cooperatives . . . 24

23.5 Families and small federations . . . 24

23.6 Policy, civic, and activist coordination . . . 24

23.7 Disaster response and field operations . . . 24

23.8 Cross-tip and forkgivening institutions . . . 24

24 Second- and Third-Order Effects 25

24.1 Members become more temporally literate . . . 25

24.2 Relays become political institutions . . . 25

24.3 Guilds become memory-producing machines . . . 25

24.4 DAOs become constitutional rather than merely financial . . . 25

24.5 Nodes become civically stronger . . . 25

24.6 Forks become more governable . . . 25

25 Risks Revealed by the Scenarios 25

26 Design Conclusions Drawn From the Scenarios 26

27 Closing Synthesis 26

A Compact Scenario Matrix **27**

B Next Expansion **28**

Abstract

This volume focuses on real protocol use rather than primarily formal architecture. Its purpose is to show how Kapnet behaves when concrete actors do concrete things under time pressure, relay scarcity, constitutional constraints, and competing institutional goals. The key claim is that a protocol only becomes believable when its objects can be placed into lived scenarios: a user opens a room, a guild admits a member, an editor requests attestations, relays decide what to carry, a DAO votes, a provider sells archival durability, a node comes back online after an outage, or many tips must be jointly observed during disagreement.

Accordingly, the document is built around worked scenarios. Each scenario names its actors, starting state, packet flow, relay decisions, time semantics, replay outcome, and higher-order consequences. The goal is not merely to show that Kapnet can express these activities, but to show what design pressures emerge once it does. Weakwork becomes visible as a real priority discipline. Submission time becomes more nuanced than ordinary wall-clock timestamps. Rooms become scope boundaries rather than cosmetic channels. Guilds become durable labor organizations. DAOs become constitutional transition machines. Relays become economic institutions. Nodes become operators of civil memory rather than passive feed consumers. Knot races become political as well as technical events.

The scenarios chosen here are intentionally varied. Some are simple and local, such as a member making a post in a room under weakwork-gated relay policy. Others are institutionally involved, such as a guild initiating a publication workflow that requires subject-matter attestation, editorial review, treasury bond posting, and final knot inclusion. Others stress cross-tip or adverse conditions: a dispute during relay fragmentation, a room fork under constitutional conflict, a node catching up after missing a knot cycle, or a provider contract enforcing high-retention archival guarantees for one class of submissions while dropping another. Taken together, these examples aim to make the protocol legible as a living coordination stack rather than a speculative vocabulary.

1. Purpose of This Volume

The previous volumes established doctrine, object models, packet families, replay semantics, and exactness-oriented protocol structure. This volume asks a different question: what does any of that feel like once actors start using it?

That question matters because protocols often look persuasive in static diagrams but become incoherent in lived use. For a system like Kapnet, which is meant to formalize the middle between settlement and speech, scenario analysis is not peripheral. It is one of the main tests of whether the design has genuine explanatory power.

The working method of this volume is therefore concrete. Each scenario tries to answer six questions:

1. Who are the actors?
2. What is the institutional setting?
3. What packets are emitted and when?
4. How do relays and nodes treat those packets?
5. What state transitions occur in rooms, guilds, DAOs, and archives?
6. What second- and third-order consequences follow?

That final question is critical. Protocols do not merely permit actions. They reshape habits, incentives, expectations, and political possibility. A relay pricing rule changes who speaks first. An attestation flow changes who becomes authoritative. A knot cadence changes how fast institutions can safely harden memory. A namespace policy changes whether plurality is preserved or erased.

2. Actor Model Used Throughout the Scenarios

To keep the scenarios coherent, this volume reuses a common set of actor types.

2.1. End users and members

An end user is an individual operator of keys, identities, preferences, subscriptions, and local node or wallet context. A member is an end user who has been admitted into one or more rooms, guilds, or DAO-governed institutional surfaces.

A user may be:

- a reader only,
- a posting member,

- a bonded participant,
- an editor,
- a validator,
- an expert attestor,
- a delegate,
- a room steward,
- a guild officer,
- a DAO constitutional operator.

2.2. Nodes

A node in these examples is not merely a passive sync process. It is a replaying observer that may also sign, cache, route, archive, verify, or sell service. Different nodes may expose very different service profiles while still replaying the same public history.

2.3. Relays

Relays are service institutions. They accept, reject, queue, delay, prioritize, archive, and forward packets according to economic and constitutional rules. Some relays are broad public carriers. Others are guild-internal. Others are premium providers specialized in archival durability, low-latency routing, or knot candidate concentration.

2.4. Rooms

Rooms are bounded discussion or workflow surfaces. They constrain scope, moderation, topic, membership, and default publication policy. A room is where local context becomes manageable.

2.5. Guilds

Guilds are durable institutional groupings that host rooms, define role ladders, maintain archives, and request or recognize expertise. They are where recurring labor and identity gain continuity.

2.6. DAOs

DAOs here are constitutional transition machines. They define who may mutate which object, under what threshold, with what bond, and with which dispute or veto channels.

2.7. Providers

Providers are specialized service actors. In the scenarios below, providers may sell:

- archival durability,
- accelerated relay,
- expert attestation routing,
- knot observation and dispute monitoring,
- multi-tip state views,
- publication indexing,
- compliance or evidence bundles for institutional users.

3. Time Semantics Used Throughout

One of the user's explicit requests was to be heavy on submission time. That matters because Kapnet has multiple notions of time, not one.

3.1. Wall-clock time

A packet may carry a local wall-clock timestamp signed by the emitter. This is useful for user interface, ordering hints, and latency diagnostics, but it is not sovereign.

3.2. Relay arrival time

Each relay may record first-seen time, queue admission time, forwarded time, and archive commit time. These are real and operationally important, but still not sovereign.

3.3. Base-block affinity time

A submission may be associated with a base-block horizon or range. This is much closer to protocol sovereignty because replay and checkpoint logic index primarily to base chronology rather than local clocks.

3.4. Braid time

Inside Kapnet, a submission may live in a braid region where local precedence is partial. Two acts can be contemporaneous for some institutional purposes even if they reached different relays in different orders.

3.5. Knot time

Once a knot claim crystallizes part of the braid, a stronger historical boundary is established. Knot time matters for publication memory, constitutional deadlines, bond unlock windows, and dispute cutoff rules.

Thesis 1. *Submission time in Kapnet is layered: signed wall-clock time helps users, relay time helps operators, base-block time helps replay, braid time helps live coordination, and knot time helps institutions remember.*

4. A Generic Scenario Template

Before turning to the full examples, it is useful to define a generic scenario template.

Scenario template
Actors. Name all users, members, stewards, relays, nodes, providers, and DAOs. **Initial state.** State the room, guild, membership, bond, relay subscription, and latest knot horizon. **Trigger.** What event begins the flow? **Packet sequence.** Which submission classes are emitted in what order? **Time layers.** What are the relevant wall-clock, relay, block-affinity, braid, and knot times? **Relay behavior.** What is queued, rejected, accelerated, or archived? **Replay result.** What state does a healthy node converge to? **Consequences.** What institutional or economic effects follow? **Failure variants.** Where can the flow go wrong, and how is the damage bounded?

5. Scenario I: A Member Posts Into a Weakwork-Gated Room

5.1. Simple room post under real relay policy

Scenario intent
A new but admitted member makes a substantial post in a guild room where free posting is allowed only at low priority, while posts with sufficient weakwork or postage are forwarded immediately to the room's main relays.

5.1.1. Actors

- **Mira**: a newly admitted guild member.
- **River Relay**: the guild's default public relay.
- **Stone Relay**: the guild's premium low-latency relay.

- **Atlas Node**: a replaying node run by another guild member.
- **Atlas Guild**: the institution hosting the room.
- **Room /atlas/economics**: the room where the post is intended to land.

5.1.2. Initial state

Mira has valid membership credentials for the guild and posting rights in the room. The room constitution specifies three queues:

1. free queue with delay and burst limits,
2. weakwork-priority queue,
3. paid-postage immediate queue.

The guild DAO does not require editorial approval for ordinary discussion posts, but the room steward has set an anti-spam policy that causes free posts from accounts younger than seven days to enter the delayed queue unless they meet a weakwork floor.

5.1.3. Trigger

Mira writes a detailed reply to an active discussion on monetary hygiene. She wants fast visibility and chooses to attach moderate weakwork rather than pay immediate postage.

5.1.4. Illustrative packet body

```
packet_class: ROOM_POST
namespace: BTC:kap:atlas:economics
destination: room://atlas/economics
author: key:mira
membership_ref: memb://atlas/7f2a
parent_submission: sub://6a0d...91c2
body_hash: b3e7...44aa
weakwork: grade=2 nonce=847221 difficulty_hint=moderate
bond: none
postage: none
wall_clock_signed: 2026-03-24T18:11:22-04:00
block_affinity: BTC@recent+0..2
visibility: guild_public
archive_hint: standard
```

5.1.5. Time layers

Time layer	Meaning in this flow
Wall-clock time	Mira signs the packet at 18:11:22 local time.
Relay arrival time	River Relay sees it at 18:11:24; Stone Relay receives a forwarded copy at 18:11:26.
Base-block affinity	The post is intended for replay near the current BTC horizon plus the next two blocks.
Braid time	The post joins the current room braid region with several near-contemporaneous replies.
Knot time	The packet becomes part of the next room-relevant knot candidate six blocks later.

5.1.6. Relay behavior

River Relay checks four things before forwarding: membership validity, room permission, packet syntax, and weakwork floor. Because Mira satisfies the weakwork floor, the submission bypasses the delayed queue and enters the room braid immediately. Stone Relay accepts the forwarded packet because Atlas Guild has an inter-relay treaty for this room.

5.1.7. Replay result

A healthy node replays the post as a valid room submission linked to the parent reply. It is visible immediately in live room state, marked as not yet crystallized until the next knot cycle. No DAO action is needed.

5.1.8. Consequences

The first-order result is small: a post appears quickly. The second-order result is more important. Mira learns that weakwork has practical value even in everyday participation. The guild preserves open discussion while still pricing urgency. The relay can remain mostly public without becoming costless spam transport.

5.1.9. Failure variant

If Mira had omitted weakwork and paid no postage, her post would still be valid but would sit in the delayed queue. The protocol consequence is not censorship but degraded priority. This is one of the core social uses of weakwork: it turns urgency into a visible choice.

6. Scenario II: A Guild Requests Expert Attestation Before Publication

6.1. Guild editorial workflow with subject-matter expertise

Scenario intent
A guild wants to publish a technical note under its official namespace. The guild constitution requires two editor signatures and one recognized subject-matter attestation before the note may be elevated from room discussion to guild publication state.

6.1.1. Actors

- **Jules**: author of the technical note.
- **Editor Rowan** and **Editor Sia**: guild editors.
- **Dr. Vale**: recognized subject-matter attestor.
- **North Relay**: editorial relay with archive guarantees.
- **Foundry Guild**: the publishing institution.
- **Foundry DAO**: constitutional governor for official publication.

6.1.2. Initial state

The room `foundry/drafts` is open for drafting, but publication into `guild://foundry/publications` requires:

1. draft submission,
2. editorial review thread,
3. subject-matter attestation packet,
4. final publication transition packet co-signed by two editors.

6.1.3. Flow

Jules first submits a draft packet to the drafting room. Editors discuss it publicly in the room. After revisions, Rowan issues an `ATTESTATION_REQUEST` packet naming Dr. Vale and attaching a compensation bond drawn from the guild editorial budget. Dr. Vale then issues an `ATTESTATION_RESPONSE` packet with one of three outcomes: endorse, endorse-with-caveat, or reject.

In this example, Dr. Vale endorses with caveat. The caveat is then answered by Jules in a revision packet. Once Rowan and Sia agree that the caveat has been adequately resolved, they

issue a co-signed `PUBLICATION_TRANSITION` packet that changes the object's state from `draft` to `guild_publication`.

6.1.4. Timeline table

Step	Packet class	Result
T0	`DRAFT_SUBMIT`	Draft enters draft room braid.
T1	`REVIEW_COMMENT` x 7	Editors and readers discuss revisions.
T2	`ATTESTATION_REQUEST`	Expert attestation formally requested, bond posted.
T3	`ATTESTATION_RESPONSE`	Expert endorses with caveat.
T4	`DRAFT_REVISION`	Author resolves caveat.
T5	`PUBLICATION_TRANSITION`	Editors co-sign promotion into official guild publication state.
T6	`KNOT_CANDIDATE`	Publication state is crystallized into a guild archive knot.

6.1.5. Replay result

A replaying node can show the complete publication lineage: original draft, review comments, attestation request, attestation response, revision, final publication packet, and knot anchor. The publication is therefore not just a static artifact but an institutionally replayable editorial history.

6.1.6. Consequences

The use case here is bigger than publishing. It shows how expertise can be made legible without making it absolute. The expert does not replace the guild. The guild does not ignore expertise. The attestation is one packet class in a wider constitutional machine.

7. Scenario III: A New Room Is Created Under a Guild Constitution

7.1. Room creation with bounded authority

Scenario intent
A guild steward wants to create a new room for a rapidly growing topic, but room creation is constitutionally constrained to prevent namespace sprawl and stealth policy fragmentation.

7.1.1. Actors

- **Steward Hana**: a guild steward with room-proposal authority.
- **Guild DAO**: grants or rejects room creation requests.
- **Members**: affected by the new room's permissions and moderation defaults.

- **Relay operators**: must decide whether to host the new room immediately or only after final approval.

7.1.2. Flow

Hana issues a `ROOM_CREATE_PROPOSAL` packet with:

- proposed room name,
- declared topic scope,
- initial moderation defaults,
- retention policy,
- visibility class,
- sponsoring guild object,
- justification text,
- requested relay class.

The Guild DAO requires two officer endorsements and a 48-hour objection window. During this window, members may submit `ROOM_OBJECTION` packets. If the objection threshold is not met, the steward and one officer issue a `ROOM_ACTIVATE` transition. Relays then instantiate the room namespace.

7.1.3. Real significance

This scenario shows that even something as mundane as creating a room can be protocol-significant. In ordinary platforms, channel creation is usually an admin toggle. In Kapnet, it can be a constitutional event with replayable rationale and accountable consequences.

7.1.4. Consequences

The immediate consequence is a new workflow surface. The deeper consequence is that institutional topology becomes historical rather than opaque. Future members can ask not only what rooms exist, but why they exist, who sponsored them, what objections were raised, and under which governance rules they were created.

8. Scenario IV: Membership Admission With Bond, Sponsorship, and Cooldown

8.1. Guild membership is not just a boolean

Scenario intent

A guild wants open growth but needs protection against drive-by abuse. It therefore uses a multi-stage membership flow: sponsorship, temporary bond, probationary rights, and delayed unlock of full permissions.

8.1.1. Actors

- **Ari**: membership applicant.
- **Sponsor Lynn**: existing member who vouches for Ari.
- **Guild DAO**: controls admission thresholds.
- **Relays**: enforce probationary rights before full activation.

8.1.2. Flow

Ari submits an `APPLICATION` packet. Lynn issues a `SPONSORSHIP` packet referencing Ari's application. The guild DAO constitution specifies that one sponsor plus one bond is enough for provisional admission, but full posting privileges in policy-sensitive rooms activate only after a cooldown measured in both wall-clock and knot-confirmed time.

8.1.3. Time importance

This scenario demonstrates why submission time cannot be reduced to one clock. The guild uses:

- wall-clock time for the nominal duration of probation,
- knot-confirmed time to avoid replay ambiguity caused by relay delays,
- member-age state to determine queue priority in certain rooms.

8.1.4. Consequences

The use case is common: professional, editorial, scientific, or activist groups often need bounded openness. Kapnet makes the stages explicit and replayable, reducing the ambiguity of who counted as a member when.

9. Scenario V: A Treasury-Bonded Proposal in a DAO-Controlled Guild

9.1. Economic seriousness for consequential proposals

Scenario intent
A member proposes an institutional change with treasury implications. The DAO requires a bond so that low-effort or malicious proposals impose a real cost.

9.1.1. Actors

- **Proposal author**: initiates the policy change.
- **Treasury DAO**: governs proposal thresholds and bond sizes.
- **Voters**: members with proposal review rights.
- **Treasury relay**: ensures high-durability handling of treasury-relevant packets.

9.1.2. Flow

The author submits a `POLICY_PROPOSAL` packet with a required bond. The proposal enters a review room. Commentary and amendments are attached as child submissions. At the close of the review window, a `VOTE_OPEN` packet fixes the text hash to avoid stealth edits. Votes are then submitted as cryptographically attributable ballots or threshold-based delegation packets, depending on DAO design.

If the proposal passes, a `POLICY_ENACT` packet updates the relevant object. If it fails by a wide margin or is shown to be maliciously malformed, part of the bond may be slashed according to constitutional rules.

9.1.3. Why this matters

The bond does not merely raise the cost of spam. It changes proposal culture. It signals that consequential institutional change should leave a trace of seriousness in both language and economics.

10. Scenario VI: A Dispute Over Moderation Escalates From Room to Guild to DAO

10.1. Moderation becomes constitutional when replayable

Scenario intent
A room steward deletes or suppresses a submission under room policy. The author disputes the action, claiming the steward exceeded local authority.

10.1.1. Actors

- **Author**: original poster whose submission was suppressed.
- **Room steward**: invoked room moderation policy.
- **Guild appeal committee**: hears mid-tier disputes.
- **DAO court room**: final constitutional venue.

10.1.2. Flow

The steward emits a `MODERATION_ACTION` packet citing rule basis and target submission. The author answers with a `DISPUTE_OPEN` packet. The guild appeal committee reviews whether the steward acted within room scope. If the committee deadlocks or the author claims the room rule itself conflicts with guild constitution, the dispute escalates to the DAO court room.

At the DAO level, the question is no longer only whether the submission was offensive, disruptive, or low quality. The question becomes: did the steward possess authority under the relevant constitutional path?

10.1.3. Consequences

Moderation stops being pure invisible discretion. That does not eliminate hard judgment. It does make the line between room policy and constitutional overreach easier to inspect later. This is a major use case for any serious digital institution.

11. Scenario VII: A Node Comes Back Online After Missing a Knot Cycle

11.1. Replay after outage

Scenario intent
A personal node was offline during a period of heavy room activity and at least one knot cycle. The user returns and wants trustworthy reconstruction without trusting a single relay's current feed.

11.1.1. Actors

- **Offline node**: the user's machine.
- **Three relays**: with partially overlapping archives.
- **Provider archive**: sells high-durability retrieval for knot-adjacent packets.

- **Current room members**: continue to post while the node is offline.

11.1.2. Flow

When the node reconnects, it first syncs base-chain state to recover the relevant block horizon. It then requests packet ranges from several relays and a provider archive, asking specifically for:

- all packets in room braid regions after its last local checkpoint,
- all knot claims touching those ranges,
- any dispute packets or invalidation notices,
- archive receipts for the accepted knot.

The node then rebuilds the braid, verifies knot candidacy and acceptance, checks receipts, and resumes live operation from the latest accepted horizon.

11.1.3. Why this matters

This is one of the strongest arguments for replayable institutional state. A user does not have to ask, “what happened while I was gone?ín a purely social sense. The node can answer structurally.

11.1.4. Second-order consequence

Nodes become less feed-dependent and more history-capable. That changes the civil balance between users and providers. Providers still matter, but they become sellers of availability and durability rather than sole owners of memory.

12. Scenario VIII: Knot Race During Relay Fragmentation

12.1. Checkpoint politics under imperfect network conditions

Scenario intent
Two clusters of relays see overlapping but not identical packet sets near the end of an active braid period. Competing knot claims are produced.

12.1.1. Actors

- **Relay cluster A**: public relays with broad but noisy connectivity.
- **Relay cluster B**: premium relays with lower latency but narrower access.

- **Independent nodes**: observe both clusters.
- **Guild archives**: want a clean canonical memory surface.

12.1.2. Flow

Each relay cluster produces or forwards a knot candidate over what it believes is the current braid frontier. Candidate A includes more total packets but lower quality attestation on some expensive packets. Candidate B includes fewer packets but higher validation confidence and stronger archive receipts.

Independent nodes must evaluate according to protocol rules, not aesthetics. They check:

1. object validity,
2. knot claim structure,
3. constitutional constraints,
4. archive and witness receipts,
5. penalties for pathological timing divergence,
6. any local policy preferences permitted by the guild.

12.1.3. Consequences

This scenario reveals that knot acceptance is not merely a compression routine. It is where social confidence, relay economics, validation labor, and historical legitimacy meet. A protocol that lacks a believable knot-race story lacks a believable memory story.

13. Scenario IX: A Publication Pipeline Across Rooms, Guild, Archive, and Indexer

13.1. From room discussion to durable public object

Scenario intent
A long-form report evolves through several surfaces before becoming durable guild output indexed for external discovery.

13.1.1. Flow overview

1. Draft enters a room.
2. Discussion and revisions occur in public or semi-public braid.

3. Editors request attestations.
4. Treasury bond is posted for publication costs.
5. Publication packet transitions object into guild publication namespace.
6. Archive provider stores a high-durability copy and returns a receipt.
7. Public indexer emits a discoverability packet under the guild's publication policy.
8. Knot candidate crystallizes the publication lineage.

13.1.2. Why this is a real use case

This is how magazines, scientific collectives, policy groups, and open research guilds actually operate. Kapnet's value is that all these transitions can become explicit and replayable without forcing the report itself to be equivalent to a settlement event.

14. Scenario X: Cross-Room Working Group Coordination

14.1. One project, many rooms, one guild memory

Scenario intent
A guild runs a multi-disciplinary project where legal, technical, editorial, and outreach rooms all contribute different packet classes to a shared project object.

14.1.1. Actors

- **Technical room**
- **Editorial room**
- **Legal room**
- **Outreach room**
- **Project steward**
- **Guild DAO**

14.1.2. Flow

Each room emits packets into its own namespace but references a common project identifier. The project steward periodically issues `PROJECT_SUMMARY` packets summarizing current state. The guild archive can later reconstruct the full cross-room history without flattening the rooms into one undifferentiated feed.

14.1.3. Consequence

This use case shows why rooms are not cosmetic channels. They are scope-preserving membranes. Cross-room coordination becomes richer because differentiation is preserved rather than collapsed.

15. Scenario XI: Multi-Tip Coordination During Divergent Policy Worlds

15.1. One institution watches several timechains

Scenario intent
A guild is Bitcoin-centric but maintains observers and mirror rooms for BTC and BCK environments. It wants to discuss, archive, and react across both without pretending the chains are identical.

15.1.1. Actors

- **BTC observers**
- **BCK observers**
- **Cross-tip room**
- **Multi-tip relay provider**
- **Guild archive**

15.1.2. Flow

Observers emit tip-scoped packets under distinct namespaces, for example `BTC:kap:obs` and `BCK:kap:obs`. A cross-tip room accepts both classes but requires that summary packets preserve source-tip labeling. Archive receipts also preserve tip distinction.

15.1.3. Why this is meaningful

The application is broader than chain monitoring. It is a model for civil coexistence under plural temporal anchors. The guild can retain one institutional memory while refusing false homogenization.

15.1.4. Third-order effect

Over time, source-tip labeling itself becomes part of the institution's interpretive literacy. Members learn to treat base plurality as normal rather than as mere chaos.

16. Scenario XII: Provider Contract for High-Retention Archival Service

16.1. Service contracts become protocol-visible

Scenario intent
A research guild needs retention guarantees for certain classes of packets but not others. It therefore contracts with an archival provider whose obligations are expressed through packets and receipts rather than private email promises.

16.1.1. Actors

- **Research guild**
- **Archive provider**
- **Ordinary relays**
- **Auditor nodes**

16.1.2. Flow

The guild emits a `SERVICE_CONTRACT_OPEN` packet specifying retention period, service class, eligible namespace family, receipt frequency, and penalty basis for non-delivery. The provider countersigns with `SERVICE_CONTRACT_ACCEPT`. Thereafter, whenever an eligible packet is archived, the provider emits a signed `ARCHIVE_RECEIPT` packet.

Auditor nodes can later challenge missing archival service by comparing expected receipts against actual retained state.

16.1.3. Consequence

Service quality becomes more observable. The provider is no longer merely trusted in a vague sense. It is trusted through a replayable receipt surface.

17. Scenario XIII: Emergency Room Fork Under Constitutional Conflict

17.1. Forkability as institutional safety valve

Scenario intent

A room becomes ungovernable because half the participants believe the steward is acting within policy and half believe the room's policy is itself illegitimate. The guild constitution permits emergency fork under specified conditions.

17.1.1. Flow

An emergency fork requires:

1. a `FORK_NOTICE` packet,
2. supporting signatures from threshold members,
3. explicit inheritance map for archives and permissions,
4. a decision on whether pending disputes migrate or terminate,
5. a namespace split marker so future nodes do not confuse post-fork room state.

17.1.2. Consequences

The obvious consequence is institutional rupture. The deeper consequence is that rupture itself can be formalized. Instead of pretending conflict did not happen, Kapnet lets the history of the split remain visible. This is a major use case for politically serious institutions.

18. Scenario XIV: A Local Family or Small Firm Uses Kapnet as Internal Coordination Memory

18.1. Not every use case is a public guild

Scenario intent

A small firm or family uses a private or semi-private Kapnet setup to coordinate responsibilities, decisions, and archival memory without needing a global public audience.

18.1.1. Use pattern

The firm runs a small relay, a few rooms, and one lightweight DAO for role assignment. Packets govern approvals, expenses, task acceptance, and decision summaries. A public anchoring cadence may still exist, but most active coordination remains private or permissioned.

18.1.2. Why this matters

The protocol is not only for giant public institutions. Its object model is also useful for adoptive firms, families, and small federations where memory and accountability matter but the scope is modest.

19. Scenario XV: A Disaster-Response Guild Coordinates Resources Under Time Pressure

19.1. High urgency, bounded trust, many actors

Scenario intent
A disaster-response guild must coordinate incoming reports, resource requests, volunteer attestations, local room updates, and archive-worthy summaries while communications are degraded.

19.1.1. Why Kapnet fits

This use case stresses almost every primitive at once:

- weakwork for urgent routing,
- rooms for geographic or functional scope,
- guild roles for validated responders,
- attestation packets for confirming reports,
- summary packets for situation memory,
- provider receipts for durable incident archives,
- replay for after-action accountability.

19.1.2. Consequence

Even if the institution is temporary and the situation urgent, the resulting memory can be made replayable rather than being lost to ad hoc chat logs.

20. Scenario XVI: A Research Lab Uses Kapnet for Peer Review and Priority Claims

20.1. Scientific coordination without platform opacity

Scenario intent
A research lab or open scientific guild wants transparent peer review, traceable priority claims, and archival publication flows without relying on hidden platform moderation or private editorial trails.

20.1.1. Flow

Drafts, comments, methods attestations, replication requests, and publication decisions all become packet classes with explicit lineage. Priority claims can be associated with block-affinity and later knot inclusion, making chronology visible without pretending that every claim is final truth merely because it was first.

20.1.2. Importance

This scenario shows that Kapnet can support knowledge institutions where the history of criticism matters as much as the artifact eventually published.

21. Scenario XVII: Node Operator Sells Weakwork-Accelerated Relay to a Guild

21.1. Small operators can sell real service

Scenario intent
A small node operator does not mine blocks or run a giant archive, but does run a reliable low-latency relay that serves one guild's urgent packet classes in exchange for subscription revenue.

21.1.1. Consequence

This is a practical monetization story. Kapnet can create a market in which small operators earn by providing narrow but real civil infrastructure: route acceleration, knot observation, queue hygiene, or high-availability relay for a defined institution.

22. Scenario XVIII: A Full Institutional Publication and Governance Cycle

22.1. Everything at once

Scenario intent

This final scenario bundles the main objects into one end-to-end flow. A member proposes a report; the guild spins up a temporary working room; relays route packets by weakwork and postage; experts attest; editors publish; the DAO updates policy based on the report; a provider archives the full lineage; a knot race occurs; nodes reconcile; a later dispute over one editorial decision is opened and resolved.

22.1.1. Narrative outline

1. Member proposes report topic under bonded packet.
2. Guild creates temporary project room.
3. Members and experts submit drafts, reviews, and attestations.
4. Relays prioritize high-value packets using weakwork and subscription class.
5. Editors co-sign final publication transition.
6. Treasury DAO enacts a related policy update referencing the publication hash.
7. Archive provider emits durable receipts.
8. Two knot candidates emerge due to fragmented relay visibility.
9. Nodes reconcile and adopt the accepted knot.
10. A later moderation dispute references the preserved editorial lineage.

22.1.2. Why this scenario matters

This is where Kapnet either feels real or not. A protocol whose primitives cannot survive this density of interaction is not yet ready for ambitious institutions. The scenario demonstrates that the value of the system lies not in any single packet class, but in the fact that all of them can inhabit one replayable field.

23. Application Atlas

The scenarios above already imply many use cases, but it is worth stating the broader map explicitly.

23.1. Publishing and editorial institutions

Magazines, zines, journals, newsletters, and collective editorial projects can use Kapnet to formalize draft lineage, review state, attestation, publication transitions, and archives.

23.2. Research labs and peer review systems

Open science, lab coordination, replication markets, claim priority, and method review are especially suitable because chronology and critique both matter.

23.3. Professional guilds and trade bodies

Guilds for law, engineering, medicine, software, design, or craft can use room and guild objects for expert rosters, attestation flows, review boards, and continuing archive memory.

23.4. Firms and cooperatives

Firms can use Kapnet as explicit coordination memory around approvals, proposals, budgets, and governance, especially where records matter but conventional enterprise software obscures power.

23.5. Families and small federations

Smaller groups can use a lighter version for responsibility tracking, decision history, and internal archive without needing public social-media logic.

23.6. Policy, civic, and activist coordination

Campaigns, civic groups, neighborhood assemblies, and federated political bodies can use replayable rooms, disputes, publication flows, and forkability as a serious alternative to opaque platform moderation and hidden document trails.

23.7. Disaster response and field operations

Because Kapnet preserves bounded object structure under time pressure, it can help in urgent coordination environments where report lineage and archive are vital.

23.8. Cross-tip and forkgivening institutions

Any institution that wants to monitor, discuss, or coordinate across BTC, BTX, BCK, KNX, and CTB-like worlds without pretending they are the same can use source-tip-preserving namespaces and archives.

24. Second- and Third-Order Effects

Protocol use always spills beyond the immediate action. Several recurring higher-order effects appear across the scenarios.

24.1. Members become more temporally literate

Participants learn the difference between posting now, being first seen, being replayable, and being crystallized. This changes expectations about urgency and legitimacy.

24.2. Relays become political institutions

Once relays price, prioritize, and archive consequential packets, they become visible sites of institutional power. That is not a bug. It is a reality worth formalizing.

24.3. Guilds become memory-producing machines

Guilds no longer merely host activity. They produce historical surfaces whose lineage can be audited later.

24.4. DAOs become constitutional rather than merely financial

Because object mutations, disputes, role grants, and publications can all be governed through DAO rules, the DAO moves closer to being an operating constitution than a treasury gadget.

24.5. Nodes become civically stronger

Replayable history reduces dependence on purely present-tense feeds. A node can challenge a platform-style summary by rebuilding the relevant sequence itself.

24.6. Forks become more governable

Formal room and guild forks make institutional schism less mystified. Conflict is not eliminated, but it is made more legible.

25. Risks Revealed by the Scenarios

The scenario-heavy method also reveals dangers.

Risk 1. *If packet taxonomies become too numerous or too subtle, ordinary members may lose intuitive understanding of what they are doing.*

Risk 2. *If relay markets centralize, nominal replayability may coexist with de facto routing oligarchy.*

Risk 3. *If knot rules are under-specified, memory politics will become arbitrary precisely where institutions need clarity.*

Risk 4. *If room and guild constitutions are too rigid, forkability becomes the only practical safety valve and routine politics turns schismatic.*

Risk 5. *If time semantics are not well communicated, users may confuse signed wall-clock time with sovereign historical order.*

26. Design Conclusions Drawn From the Scenarios

Design Imperative 1. *Every major object class should have at least one ordinary use case and one stressed use case. If it only shines under ideal conditions, it is not mature enough.*

Design Imperative 2. *Submission time should always be exposed in layered form where relevant: local signed time, relay first-seen time, block-affinity time, and knot confirmation time.*

Design Imperative 3. *Room and guild interfaces should present institutional lineage, not only current state. A member should be able to inspect why an object or decision exists, not merely that it exists.*

Design Imperative 4. *Relay providers should expose service class, queue policy, archive policy, and receipt discipline as protocol-visible surfaces wherever possible.*

Design Imperative 5. *End-user flows should make weakwork, postage, bond, and visibility choices intelligible at the moment of submission.*

Design Imperative 6. *Knot races and replay recovery must be treated as first-class user journeys rather than edge cases for experts only.*

27. Closing Synthesis

The strongest way to understand Kapnet is not as a bag of primitives but as a world in motion. Someone signs. A relay admits. A room receives. A guild debates. An expert attests. A DAO governs. A provider stores. A node replays. A knot hardens. A dispute opens. A fork occurs. A memory remains.

That is what this volume has tried to render.

The scenarios show that Kapnet can be understood as a constitutional coordination stack for real institutions rather than a merely decorative vocabulary. A member post in a room already reveals the importance of weakwork, relay policy, layered time, and queue discipline. An editorial

publication flow reveals how expertise, review, and archives can become replayable without collapsing into settlement. Membership flows show how bonds and cooldowns create temporal and economic seriousness. Dispute flows show that moderation becomes inspectable when actions are packetized. Knot races show that memory is political as well as technical. Multi-tip scenarios show that plurality can be preserved without civil blindness.

The most important conclusion is therefore not a single mechanism. It is a change in the level at which the protocol is imagined. Kapnet is not just a way to send structured messages. It is a way to make institutional life itself more replayable, more accountable, more explicitly timed, and more economically legible.

For systems built over Bitcoin or around Bitcoin-descended civil worlds, that is not a minor add-on. It is the difference between living in chat exhaust and living in constitutional memory.

A. Compact Scenario Matrix

Scenario	Primary objects	Main time surface	Main consequence
Member room post	Submission, room, relay	relay + braid time	urgency priced without censorship
Expert publication	guild, attestation, archive	knot time	editorial lineage becomes replayable
Room creation	guild, DAO, room	objection window + knot horizon	topology gains history
Membership admission	membership, bond, cooldown	wall-clock + knot	rights become staged and explicit
Treasury proposal	DAO, proposal, bond	voting window + enactment horizon	institutional change gains economic seriousness
Moderation dispute	room, guild, DAO court	appeal deadline + knot record	moderation becomes constitutional
Node recovery	node, relays, archive, knot	block horizon + replay window	memory survives outage
Knot race	relays, knots, archives	braid frontier + knot acceptance	checkpoint legitimacy becomes legible
Cross-room project	rooms, project object, archive	summary cadence	differentiation preserved inside one project
Multi-tip observation	tip namespaces, observers, archive	per-tip block horizon	plurality remains explicit
Archive contract	provider, receipts, guild	retention period + receipt cadence	service quality becomes auditable
Emergency fork	room, guild, namespace split	fork notice horizon	schism becomes historically structured

B. Next Expansion

A natural next pass after this volume would combine these scenarios with:

1. exact packet examples for every scenario,
2. corresponding UI wireframes,
3. replay pseudocode traces,
4. failure injections for adversarial relays,
5. cost models for weakwork, postage, and archival service,
6. multi-tip operator dashboard walkthroughs.